

JAVA INTERVIEW PREPARATION

CHAPTER 2

EQUALITY AND HASHING: CORRECT JAVA VALUE SEMANTICS

Senior / Lead Java Developer Textbook Edition

Full reading material - not summarized

Equality and Hashing: Designing Correct Java Value Semantics

Senior / Lead Java Developer Reading Chapter

Java developers use `equals()` and `hashCode()` constantly, often without calling either method directly. Collections such as `HashMap`, `HashSet`, and `ConcurrentHashMap` rely on them. Testing libraries use equality to compare expected and actual values. Caches, deduplication logic, persistence entities, and domain value objects all depend on a correct definition of sameness.

The methods are small, but the decision behind them is architectural: what makes two instances represent the same thing?

For a senior developer, implementing `equals()` is not primarily a code-generation exercise. It requires understanding identity, object lifecycle, mutability, inheritance, and the behavior of every collection that will store the object.

Reference Identity and Logical Equality

Java has two important meanings of "the same."

Reference identity means that two variables point to the exact same object. It is tested with `==`.

```
CustomerNumber first = new CustomerNumber("C-100");
CustomerNumber second = new CustomerNumber("C-100");
CustomerNumber alias = first;

first == alias;    // true: both references point to one object
first == second;  // false: two separately created objects
```

Logical equality means that two objects represent the same value according to the rules of their type. It is tested with `equals()`.

```
first.equals(second); // should be true for a value object
```

The distinction can be pictured as follows:

```
first -----> CustomerNumber("C-100") <----- alias
second -----> CustomerNumber("C-100")

first == alias      true
first == second     false
first.equals(second) true when value equality is implemented
```

The default implementation inherited from `Object` behaves like reference identity. Two separately created objects are unequal unless a class overrides `equals()`.

That default is appropriate for some objects. A thread, an open network connection, or a mutable workflow instance may be meaningful primarily as one particular runtime object. A money amount, coordinate, date, or customer number is normally meaningful by value.

The first design decision is therefore not how to implement equality. It is whether the type should have value equality at all.

The Complete equals() Contract

The Java contract requires `equals()` to behave as an equivalence relation.

- Reflexive: for every non-null reference `x`, `x.equals(x)` must be true.
- Symmetric: if `x.equals(y)` is true, then `y.equals(x)` must also be true.
- Transitive: if `x` equals `y` and `y` equals `z`, then `x` must equal `z`.
- Consistent: repeated comparisons must return the same result while the relevant state remains unchanged.
- Non-null: `x.equals(null)` must return false.

These rules allow collections and application code to reason about equality predictably.

A basic value-object implementation is:

```
public final class CustomerNumber {  
    private final String value;  
  
    public CustomerNumber(String value) {  
        if (value == null || value.isBlank()) {  
            throw new IllegalArgumentException(  
                "Customer number is required"  
            );  
        }  
  
        this.value = value.trim().toUpperCase(Locale.ROOT);  
    }  
  
    public String value() {  
        return value;  
    }  
  
    @Override  
    public boolean equals(Object other) {  
        if (this == other) {  
            return true;  
        }  
  
        if (!(other instanceof CustomerNumber that)) {  
            return false;  
        }  
  
        return value.equals(that.value);  
    }  
  
    @Override  
    public int hashCode() {  
        return value.hashCode();  
    }  
}
```

The identity check is a fast path. It is not required for correctness because comparing the field would also succeed, but it avoids unnecessary work when both references are identical.

The type check rejects null and objects of unrelated types. Pattern matching introduces a correctly typed variable only when the comparison is valid.

The final line compares the field that defines logical identity. The constructor normalizes the representation, so `"c-100"` and `" C-100 "` become the same customer number. Normalization belongs in construction rather than inside every equality call because equality should be simple, predictable, and inexpensive.

Understanding the hashCode() Contract

A hash code is not a unique object identifier. It is a compact integer used to divide objects into candidate groups.

The critical rule is:

If two objects are equal according to equals(), they must return the same hash code.

The reverse is not required. Two unequal objects may have the same hash code because an `int` cannot uniquely represent every possible Java object.

```
x.equals(y) == true implies x.hashCode() == y.hashCode()
x.hashCode() == y.hashCode() does not imply x.equals(y) == true
```

A collision occurs when unequal keys produce the same hash. Hash collections are designed to handle collisions by checking equality after locating the candidate bucket.

The same fields should normally participate in both methods. Suppose equality compares `country` and `number`, but the hash uses only `number`. The implementation remains correct, although it may create unnecessary collisions. If the hash uses a field that equality ignores, equal objects may produce different hashes and the implementation becomes incorrect.

```
public final class TaxIdentifier {

    private final String country;
    private final String number;

    @Override
    public boolean equals(Object other) {
        if (this == other) return true;
        if (!(other instanceof TaxIdentifier that)) return false;

        return country.equals(that.country)
            && number.equals(that.number);
    }

    @Override
    public int hashCode() {
        return Objects.hash(country, number);
    }
}
```

`Objects.hash()` is clear and convenient. In a very hot path it may allocate or perform more work than a hand-written calculation, depending on the JDK and optimization context. That is a profiling decision, not a reason to avoid it in ordinary domain code.

How HashMap Uses Both Methods

Understanding lookup makes the contract easier to remember.

```
map.get(key)
  |
  v
calculate and spread key.hashCode()
  |
  v
select one table bucket
  |
  v
compare stored hash values
  |
  v
call equals() on candidate keys
  |
  v
return matching value or report no mapping
```

The hash code narrows the search. Equality confirms the final match.

Consider this broken class:

```
public final class ProductCode {

    private final String value;

    @Override
    public boolean equals(Object other) {
        return other instanceof ProductCode that
            && value.equals(that.value);
    }

    // hashCode() is not overridden
}
```

Two `ProductCode` instances can be logically equal while inheriting different identity-based hashes from `Object`.

```
ProductCode stored = new ProductCode("P-42");
ProductCode lookup = new ProductCode("P-42");

Map<ProductCode, Product> products = new HashMap<>();
products.put(stored, product);

stored.equals(lookup); // true
products.get(lookup); // may return null
```

The map may search a different bucket and never reach the equality comparison. This is why overriding `equals()` without overriding `hashCode()` is not a harmless omission.

Overriding only `hashCode()` is also incomplete. Equal hashes merely place objects in the same candidate group; the inherited identity-based `equals()` still treats separate objects as unequal.

The Mutable-Key Failure

Even a correctly implemented pair of methods can fail when identity-defining state changes while an object is stored in a hash collection.

```

public final class MutableCustomerKey {

    private String number;

    public void setNumber(String number) {
        this.number = number;
    }

    @Override
    public boolean equals(Object other) {
        return other instanceof MutableCustomerKey that
            && Objects.equals(number, that.number);
    }

    @Override
    public int hashCode() {
        return Objects.hashCode(number);
    }
}

```

Now consider the lifecycle:

```

MutableCustomerKey key = new MutableCustomerKey("C-100");

Map<MutableCustomerKey, Customer> customers = new HashMap<>();
customers.put(key, customer);

key.setNumber("C-200");

customers.get(key); // may return null

```

The entry remains physically stored in the bucket selected using the old hash. Lookup calculates a new hash from "C-200" and searches another bucket. The key has not vanished from memory, but it has become unreachable through normal lookup.

This defect often appears in production as an inexplicable cache miss, duplicate processing, or a Set that appears to contain an item during iteration but reports false from `contains()`.

The safest rule is:

Keys used in hash collections should be immutable, and the state participating in equality must remain stable for the complete time the key is stored.

Equality and Inheritance

Inheritance makes value equality difficult because a subtype can introduce additional state.

Suppose `Point` uses coordinates for equality:

```
public class Point {

    private final int x;
    private final int y;

    @Override
    public boolean equals(Object other) {
        return other instanceof Point that
            && x == that.x
            && y == that.y;
    }

    @Override
    public int hashCode() {
        return Objects.hash(x, y);
    }
}
```

A subtype adds color:

```
public class ColoredPoint extends Point {

    private final Color color;

    @Override
    public boolean equals(Object other) {
        return other instanceof ColoredPoint that
            && super.equals(that)
            && color.equals(that.color);
    }
}
```

A plain `Point` may consider a `ColoredPoint` equal because it sees matching coordinates. The `ColoredPoint` rejects the plain point because it also requires a color. Symmetry is broken.

Changing the parent to compare exact classes with `getClass()` prevents cross-class equality:

```
if (other == null || getClass() != other.getClass()) {
    return false;
}
```

This can be appropriate for final-style value semantics, but it has consequences for frameworks that generate proxies or subclasses. Another pattern, sometimes called `canEqual`, allows subclasses to participate explicitly, but it adds complexity and is easy to misuse.

For most domain value objects, the clearest solution is to avoid extensible equality. Make the class final or use a record. Use composition when additional information must be associated with the value.

Records and Value Equality

A record automatically implements equality and hashing from all record components.

```
public record Money(
    BigDecimal amount,
    Currency currency) {

    public Money {
        Objects.requireNonNull(amount);
        Objects.requireNonNull(currency);

        amount = amount.stripTrailingZeros();
    }
}
```

Two `Money` records are equal when their normalized amounts and currencies are equal. The compact constructor is important because `BigDecimal.equals()` considers both numerical value and scale: `1.0` is not equal to `1.00`, even though `compareTo()` returns zero. Normalizing the amount makes the equality semantics match the intended domain rule.

Records are only shallowly immutable. A record component can refer to a mutable list or array. If that component changes, equality and hashing also change. Defensive copying is still required.

```
public record Route(List<String> stops) {  
  
    public Route {  
        stops = List.copyOf(stops);  
    }  
}
```

The generated methods are convenient only after the component model correctly represents identity.

Equality for JPA Entities

Persistence entities have a lifecycle that makes equality more complicated than ordinary value objects.

An entity may be created before the database assigns its primary key:

```
Order order = new Order(); // id is null  
repository.save(order);    // id may now be assigned
```

If `equals()` and `hashCode()` depend on the generated ID, the hash code can change after persistence. If the transient entity was already stored in a `HashSet`, it can suffer the mutable-key failure.

Using every mutable database column is usually worse because ordinary business updates change equality. Using reference identity avoids that instability but means two entity instances representing the same database row are unequal across persistence contexts.

Possible strategies include:

- Use an immutable, application-assigned identifier such as a UUID from construction time.
- Use a truly stable natural business key when the domain guarantees one.
- Use generated-ID equality carefully, avoid transient entities in hash collections, and document lifecycle behavior.
- Keep value semantics in separate immutable value objects rather than forcing the entity to behave like one.

JPA providers may create proxy subclasses for lazy loading. An exact `getClass()` comparison can therefore reject a proxy representing the same row. Provider-aware class handling may be required. There is no universal entity-equality template that is correct for every identity strategy and persistence provider.

The senior decision starts with the entity lifecycle: when does identity exist, can it change, can proxies appear, and will instances cross persistence contexts?

Equality and Ordering

Sorted collections use `compareTo()` or a `Comparator`, not `hashCode()`.

`TreeSet` considers two elements duplicates when comparison returns zero. `HashSet` uses `equals()` and `hashCode()`.


```
BigDecimal first = new BigDecimal("1.0");
BigDecimal second = new BigDecimal("1.00");

first.equals(second);    // false
first.compareTo(second); // 0
```

As a result:

```
new HashSet<>(List.of(first, second)).size(); // 2
new TreeSet<>(List.of(first, second)).size(); // 1
```

Java recommends that natural ordering be consistent with equality, meaning `compareTo()` returns zero exactly when `equals()` returns true. Some standard types intentionally differ, so a senior developer must understand the collection semantics rather than assuming every Set defines uniqueness identically.

Arrays, Floating Point, and Other Traps

Arrays inherit identity-based `equals()` from `Object`.

```
byte[] first = {1, 2, 3};
byte[] second = {1, 2, 3};

first.equals(second);    // false
Arrays.equals(first, second); // true
Arrays.deepEquals(a, b); // nested arrays
```

An array used as a record component also receives array identity semantics from the generated record `equals()`. Defensive copying protects mutation but does not automatically create content equality. A dedicated value type may be clearer.

Floating-point equality requires a domain decision about NaN, signed zero, and tolerance. A tolerance-based comparison is often unsuitable for `equals()` because it can violate transitivity. Approximate comparison is normally expressed as a separate operation rather than the object's equivalence relation.

Case-insensitive equality should use a locale-independent normalization strategy when the value is an identifier. Avoid rebuilding normalized strings on every equality call.

Testing the Contract

Equality tests should create independent instances and verify the complete relation, not merely call an object against itself.

```
@Test
void customerNumberHasValueEquality() {
    CustomerNumber x = new CustomerNumber("c-100");
    CustomerNumber y = new CustomerNumber("C-100");
    CustomerNumber z = new CustomerNumber(" C-100 ");

    assertEquals(x, x);    // reflexive
    assertEquals(x, y);    // x -> y
    assertEquals(y, x);    // symmetric
    assertEquals(y, z);
    assertEquals(x, z);    // transitive
    assertEquals(x.hashCode(), y.hashCode());
    assertNotEquals(x, null);
    assertNotEquals(x, "C-100");
}
```

A collection-level test confirms the behavior that matters to callers:

```
@Test
void equalCustomerNumberFindsHashMapEntry() {
    Map<CustomerNumber, String> map = new HashMap<>();
    map.put(new CustomerNumber("C-100"), "Alice");

    assertEquals(
        "Alice",
        map.get(new CustomerNumber("c-100"))
    );
}
```

Libraries such as `EqualsVerifier` can check many contract details automatically, but generated verification does not decide which fields should define domain identity. That remains a design responsibility.

Production Perspective

Incorrect equality frequently appears as a problem in another subsystem.

A cache reports misses for values that look identical. A deduplication Set processes the same business event twice. A JPA entity cannot be removed from a collection after receiving an ID. A TreeSet silently drops a value that a HashSet retains. A mutable key is visible during iteration but cannot be found with `contains()`.

These incidents are difficult because logs often print identical field values while the runtime behavior depends on object identity, an old bucket location, proxy class differences, or inconsistent ordering.

When diagnosing such a problem, inspect:

- The exact fields used by `equals()`, `hashCode()`, and ordering.
- Whether those fields changed after insertion.
- Whether the compared objects have different runtime classes or proxies.
- Whether normalization occurs consistently at construction.
- Whether the collection is hash-based, sorted, identity-based, or concurrent.
- Whether entity identifiers exist before the object enters the collection.

The solution is not to add more fields until a test passes. The solution is to define identity from the domain lifecycle and make that identity stable.

Interview-Ready Summary

`equals()` defines logical equality, while `==` tests reference identity. The equality contract requires reflexivity, symmetry, transitivity, consistency, and a false result for null.

`hashCode()` narrows hash-collection lookup to a candidate bucket. Equal objects must have equal hash codes, although unequal objects may collide. The same stable identity fields should normally participate in both methods.

Hash collections first use the hash and then use equality. Overriding only one method breaks that process. Mutating identity fields after insertion can make an entry unreachable because it remains in the bucket selected by its old hash.

Inheritance complicates equality because subtypes can add state and break symmetry. Final value classes and records are usually safer. Records generate value methods from all components but remain only shallowly immutable.

JPA entities require lifecycle-specific reasoning because generated IDs, transient state, persistence contexts, and proxies affect identity. Sorted collections introduce another equality definition through comparison, so natural ordering should normally be consistent with `equals()`.

A senior implementation begins by defining what the object represents, when its identity becomes available, whether that identity can change, and which collections or frameworks will use it.

Revision Notes

- `==` compares references; `equals()` compares logical values when overridden.
- The default `Object.equals()` uses reference identity.
- `equals()` must be reflexive, symmetric, transitive, consistent, and false for null.
- Equal objects must have equal hash codes.
- Equal hashes do not guarantee equal objects; collisions are expected.
- `HashMap` uses the hash to choose a bucket and `equals()` to confirm a key.
- Override `equals()` and `hashCode()` together.
- Use the same stable identity fields in both methods.
- Do not mutate identity fields while an object is stored in a hash collection.
- Final value classes and records avoid many inheritance equality problems.
- Records provide shallow, component-based equality.
- JPA entity equality must match identifier lifecycle and proxy behavior.
- `TreeSet` uniqueness comes from comparison; `HashSet` uniqueness comes from equality and hashing.
- Arrays require `Arrays.equals()` or a dedicated value wrapper for content equality.
- Test the full equality contract with independent instances and collection behavior.